COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	September 11, 2026	**DATE SUBMITTED:**	September 11, 2026
NAME:	Jan Mikaela Guarin Jiro Luis Katindig Juan Paulo Lara Gian Carlo G. Libo-on Ramelissa Melith V. Limpio	**INSTRUCTOR:**	Engr. Jimlord Quejado

SEATWORK 8.1 **Using Sorting Algorithms Pt. 2**

1. PROCEDURE OUTPUT

1. Explain the quick sort, shell sort, and merge sort types of sorting algorithms.

Quick Sort: This algorithm chooses one element in the array called a “pivot” and divides the array into two parts based on the pivot. Using this pivot, the elements lesser than the pivot are moved to the left side of the list and those greater than the pivot are moved to the right side. And so, the list is divided into two sublists and recursively sorts the sublists until the conditions are satisfied (j > pivot, j < pivot, and j == pivot).

Shell Sort: Similar to insertion sort, instead of comparing numbers right next to each other, it compares numbers that are far apart first. It uses a sequence of decreasing “gap” values where elements separated by the current gap are sorted using insertion sort. After each pass, the gap is reduced, and this continues until the gap becomes 1, at which point the array is fully sorted.

Merge Sort: Divides the arrays into parts until it reaches two comparable values. After that, the lowest values in the smallest group are compared to another group. Then it is inputted in a new sorted array that lists the values from least to greatest.

2. Give simple sample programs in C++ that use the above sorting algorithms. Use a user input of 10 integer values in your example elements in an array to be sorted. Explain how the programs work.

Quick Sort:

quick_sort.cpp > main()

```cpp
#include <iostream>
using namespace std;

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivot = arr[high];  // Choose last element as pivot
        int i = low - 1;

        for (int j = low; j < high; j++) {
            if (arr[j] < pivot) {
                i++;
                swap(arr[i], arr[j]);
            }
        }
        swap(arr[i + 1], arr[high]);  // Place pivot in correct position
        int pi = i + 1;

        quickSort(arr, low, pi - 1);   // Sort left part
        quickSort(arr, pi + 1, high);  // Sort right part
    }
}

int main() {
    int arr[10];

    cout << "Enter 10 integers: ";
    for (int i = 0; i < 10; i++)
        cin >> arr[i];

    quickSort(arr, 0, 9);

    cout << "Sorted array: ";
    for (int i = 0; i < 10; i++)
        cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

Explanation:

Quick sort picks a pivot (in this case, the last element of the current range) and cuts the array in half so everything smaller than the pivot ends up to its left, everything larger to its right. The pivot is then in its final sorted position. This partition step is done by the for loop with a pointer i tracking the boundary of "smaller than pivot" elements. The function then recursively applies the same process to the left and right sub-arrays until each range has 0 or 1 elements.

Shell Sort:

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[10];

    cout << "Enter 10 integers: ";
    for (int i = 0; i < 10; i++) {
        cin >> arr[i];
    }

    for (int gap = 10 / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < 10; i++) {
            int temp = arr[i];
            int j = i;

            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }

            arr[j] = temp;
        }
    }

    cout << "Sorted array: ";
    for (int i = 0; i < 10; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Explanation:

The Shell Sort code first asks the user to enter 10 integers and stores them in an array. It then starts with a gap of 5 and compares elements that are five positions apart. If a larger value is found before a smaller value, the larger value is moved to the right and the smaller value is placed in its proper position. The gap is then reduced from 5 to 2 and finally to 1. When the gap becomes 1, the algorithm makes its final comparisons and completely sorts the array from smallest to largest.

Merge Sort:

```cpp
#include <iostream>

void merge(int arr[], int left, int mid, int right) {
    int temp[10];
    int i = left, j = mid + 1, k = left;

    while (i <= mid && j <= right) {
        if (arr[i] < arr[j]) {
            temp[k] = arr[i];
            i++;
        } else {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }

    while (i <= mid) {
        temp[k] = arr[i];
        i++;
        k++;
    }

    while (j <= right) {
        temp[k] = arr[j];
        j++;
        k++;
    }
    for (int i = left; i <= right; i++) {
        arr[i] = temp[i];
    }
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

```cpp
int main() {
    int arr[10];

    std::cout << "Enter 10 integers: ";
    for (int i = 0; i < 10; i++) {
        std::cin >> arr[i];
    }

    mergeSort(arr, 0, 9);

    std::cout << "Sorted array: ";
    for (int i = 0; i < 10; i++) {
        std::cout << arr[i] << " ";
    }

    return 0;
}
```

This sample merge sorting code uses the middle value to divide the list into two arrays with the same size. Considering that the array has 10 elements, this array is divided into two, resulting in two arrays with five elements. Then, each subarray is sorted by recursively using the merge sort function. This division continues after each array has the number of elements of zero or one. After this, these arrays are merged in the sorted order by sorting the sub-arrays with the size of one or two to form sorted subarrays with the size of two or three, which are then sorted again to form subarrays with the size of five. Then, these sub-arrays are sorted and combined into a complete sorted array.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.